Chapter 11 — Typography, Colour and Visual Effects

Layout decides *where* boxes go; this chapter is how they *look*. Most of a page is text, so typography — fonts, sizes, line spacing, line length — does more for readability than any other styling. Then colour and contrast (which decide whether some readers can read at all), and the decorative toolkit: backgrounds and gradients, borders and radius, shadows, filters, blend modes (rules for mixing an element's colours with what's behind it), clipping, masks and image fitting — each with a performance or accessibility side that's easy to miss.

In this chapter: 11.1 Fonts, `@font-face`, variable fonts · 11.2 Loading fonts well (`font-display`, preload, subsetting, self-hosting) · 11.3 Text properties · 11.4 Colour and contrast, `color-mix()` and relative colours · 11.5 Backgrounds and gradients · 11.6 Borders, radius and outlines · 11.7 Shadows · 11.8 Filters, blend modes, clipping and masks · 11.9 `object-fit` and `object-position`.

As in Chapters 7–10, every example is a page in `code/`, screenshotted by `code/make_figures.mjs`; the `NN_*.mjs` scripts use `code/measure.mjs` (a copy of Chapter 10's helper) to read back what Chrome computed — font load status, contrast ratios, even individual **pixel colours** from screenshots to check gradient and blend-mode maths. The one web font, **Inter**, is in `code/media/` with its SIL Open Font License (free to use and redistribute).

11.1 Fonts

What it is

A **font family** is a designed set of letter shapes (**glyphs**), usually in several **weights** (thin → black) and **styles** (normal, italic):

```
body {
  font-family: "Inter", system-ui, -apple-system, "Segoe UI", Roboto, sans-serif;
  font-size: 1rem;           /* 16px by default */
  font-weight: 400;          /* 100–900; normal = 400, bold = 700 */
  font-style: normal;        /* italic | oblique */
  line-height: 1.5;
}
/* shorthand: [style] [variant] [weight] [stretch] size[/line-height] family  (size and family required) */
h1 { font: italic 700 2rem/1.2 Georgia, serif; }
```

Font stacks and generic families

`font-family` is a **fallback list**: for every character the browser uses the first family that's available *and* has that glyph. Always end with a **generic family**, which always maps to something installed:

Generic	Looks like	Typical mapping
`serif`	strokes with small "feet"	Times New Roman
`sans-serif`	no feet	Helvetica / Arial
`monospace`	every character equally wide	Menlo, Consolas, Courier
`system-ui`	the OS UI font	San Francisco (Apple), Segoe UI (Windows), Roboto (Android)
`cursive`, `fantasy`	script / decorative	varies wildly — avoid for body text
`ui-serif`, `ui-sans-serif`, `ui-monospace`, `ui-rounded`	the OS's UI variants	only some platforms (Apple mainly)

A **system font stack** (`system-ui, …, sans-serif`) costs zero downloads and looks native — GitHub and many design systems use one. Quote family names with spaces; never quote generic names (`"serif"` means a

font literally named "serif").

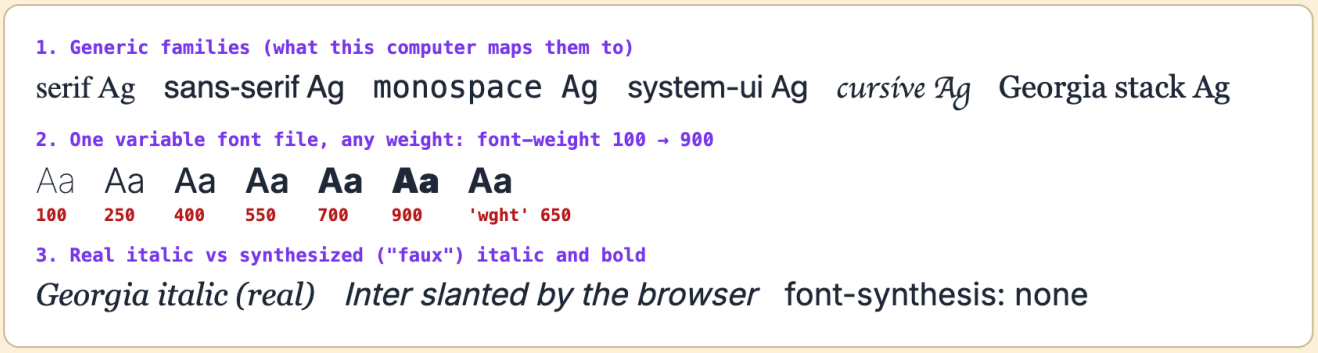


Figure 11.1 — `code/01_fonts.html`: generic families on this Mac, one variable font at seven weights, and real vs synthesized italic.

`@font-face`: using your own font

```
@font-face {
  font-family: "Inter Var";                                  /* the name YOU choose */
  src: url("media/inter-latin-wght-normal.woff2") format("woff2");
  font-weight: 100 900;                                      /* this file covers a range */
  font-style: normal;
  font-display: swap;                                        /* 11.2 */
}
.inter { font-family: "Inter Var", system-ui, sans-serif; }
```

1. `@font-face` only *declares* a font; nothing downloads yet.
2. When text needs that family (at a matching weight/style), the browser **downloads the file** — lazily.
3. Per weight/style request it picks the rule whose descriptors (`font-weight`, `font-style`, `font-stretch` inside `@font-face`) match best. With **static fonts** (one weight and style per file) you write one rule per file (Regular 400, Bold 700, Italic…) sharing one `font-family` name; `font-weight: 100 900` declares a variable font covering the range.
4. If no face matches a requested bold or italic, the browser **synthesizes** one (smears glyphs for bold — **faux bold** — slants for italic). `font-synthesis: none` turns this off (last sample in Figure 11.1 stays upright).

Font formats

Format	Use
WOFF2	The only one you need: Brotli-compressed, supported by every current browser
WOFF	Older compressed format; very old browsers only
TTF / OTF	Desktop formats; larger; avoid on the web
EOT, SVG fonts	Obsolete

Variable fonts

A **variable font** stores one or more **axes** of variation in one file, so every weight from 100 to 900 (not just nine steps) comes from one download:

Axis tag	Meaning	Prefer this property
`wght`	weight	`font-weight: 550`
`wdth`	width	`font-stretch: 85%`
`ital`	italic on/off	`font-style: italic`
`slnt`	slant angle	`font-style: oblique 10deg`
`opsz`	optical size (shapes tuned for small vs large text)	`font-optical-sizing: auto` (default)

`font-variation-settings: "wght" 650, "GRAD" 50` sets axes directly — needed for **custom axes** (uppercase tags like `GRAD`, grade) — but overrides the higher-level properties and doesn't compose through inheritance (a child setting one axis wipes its parent's), so use normal properties where they exist. Baseline widely available since 2018.

`code/01_fonts.mjs` confirms the one file serves the whole range with genuinely different outlines:

```
Inter Var weight=100 900 status=loaded display=swap
document.fonts.check("700 22px Inter Var"): true
"Hamburgefonstiv" at 40px, width by weight -> 100: 307.5px, 400: 325.2px, 700: 340.5px, 900: 353.5px
font-synthesis on the last sample: none
```

("Hamburgefonstiv" is a classic type-testing word.) Heavier weights are wider — real glyphs, not a smeared regular.

Trade-offs

Option	+	−
System font stack	0 bytes, instant, native feel	Looks different per OS; less brand identity
One variable web font	Any weight from one file (~50 KB here for Latin)	One larger file; loading to manage (11.2)
Several static web fonts	Each file small	Every extra weight/style is another request

Common mistakes

- No generic family at the end — a failed web font falls back to the browser default (often Times).
- `font-weight: 600` with only 400 and 700 files → the browser picks 700 (nearest heavier); `bold` on a family with no bold face → faux bold (provide the face or `font-synthesis: none`).
- Loading five weights "just in case".
- Quoting generic names (`"sans-serif"`).

11.2 Loading fonts well: `font-display`, preload, subsetting

The problem it solves

A web font is a separate download that usually starts **late** — once HTML and CSS are parsed and the browser knows the text needs it. Meanwhile:

- **FOIT** (Flash Of Invisible Text): text hidden until the font arrives — blank space on slow networks.
- **FOUT** (Flash Of Unstyled Text): fallback font shown, then **swapped** — the reflow may shift content (hurting **CLS**, Cumulative Layout Shift, a Core Web Vital — Chapter 14).

`font-display`

This descriptor (CSS Fonts Level 4; Baseline widely available) sets three periods:

1. **Block period** — text drawn invisibly while the font loads.
2. **Swap period** — text drawn in the fallback; if the font arrives, it swaps in.
3. **Failure period** — too late: keep the fallback for this page.

Value	Block period	Swap period	Use for
`auto`	browser decides (usually like `block`)		—
`block`	short (spec suggests 3 s)	infinite	icon fonts (characters are pictures), where fallback glyphs would be wrong
`swap`	extremely short (≈0–100 ms)	infinite	brand/heading fonts that must appear
`fallback`	extremely short	short (≈3 s)	body text: use the font only if it comes quickly
`optional`	extremely short (≈100 ms)	**none**	body text where stability matters most; used from cache on later pages

`code/02_font_display.html` declares one font three times (`block`, `swap`, `optional`, with a Georgia fallback so you can see which is used). `code/02_font_display.mjs` intercepts the network and delays every font by 1.5 s (timings machine-dependent):

```
t≈301ms  F-block=loading  F-swap=loading  F-optional=loading
t≈3503ms  F-block=loaded  F-swap=loaded  F-optional=loaded
  .block     drawn with: Menlo, Inter (web font)
  .swap      drawn with: Menlo, Inter (web font)
  .optional  drawn with: Georgia, Menlo
```

(Menlo is the red `<code>` labels' font.) The `optional` font **did** download, but its window had closed, so the paragraph stays Georgia for this view.



Figure 11.2 — FOIT vs FOUT vs "no swap", captured by delaying the font download.

Making fonts arrive sooner

Technique	How	Why
Preload the critical font	`<link rel="preload" href="/fonts/inter.woff2" as="font" type="font/woff2" crossorigin>`	Starts downloading during HTML parsing instead of after CSS. Font requests are always CORS-mode (Cross-Origin Resource Sharing), so `crossorigin` is required even same-origin — without it the preload doesn't match the real request and the font downloads twice. Preload only 1–2 fonts.
Self-host	Serve files from your own domain	No extra DNS/TLS connection (System Design Fundamentals); no third-party privacy issue (January 2022, Munich Regional Court — LG München I, case 3 O 17493/20 — held that loading Google-hosted fonts sent visitors' IP addresses to Google without consent, breaching the GDPR, and awarded the visitor €100)
Subset	Keep only needed characters; declare `unicode-range: U+0000-00FF, …`	The Inter file here is only the Latin subset (48 KB). A `unicode-range` face downloads only if the page uses one of its characters.
Match the fallback's metrics (letter sizes, line spacing)	`@font-face { font-family: "Inter Fallback"; src: local("Arial"); size-adjust: 107%; }`	The swap barely moves text, reducing CLS. `size-adjust` works in all current browsers (Safari since 17); `ascent-override` / `descent-override` not yet in Safari.
Fewer files	One variable font instead of 4–6 static ones	Fewer requests
Cache	Long `Cache-Control` max-age with hashed file names (fingerprinted, so new versions get new names)	Later pages are instant

From JavaScript

The **CSS Font Loading API** (`document.fonts`) exposes each face (`status: "unloaded" | "loading" | "loaded" | "error"`), `document.fonts.load('700 1em Inter')` to start a load, and `document.fonts.ready` (a promise), which the chapter scripts await before measuring.

In the interview

> **30-second answer:** "Web fonts load late, so text either goes invisible (FOIT) or shows a fallback then swaps (FOUT), possibly shifting layout. I choose `font-display`: `swap` for brand text, `optional` or `fallback` for body text, `block` only for icon fonts. I self-host WOFF2, subset with `unicode-range`, preload the one or two critical files with `crossorigin`, use one variable font instead of many weights, and tune the fallback with `size-adjust` to reduce CLS."

- *When is `optional` better than `swap`?* → When layout stability matters more than showing the brand font on first view; it's cached and used on the next page.

11.3 Text properties

The essentials

Property	Example	Notes
`font-size`	`1.125rem`	`rem` so user settings work (Chapter 6); fluid with `clamp()` (§10.7)
`line-height`	`1.5`	**Unitless** (below). Body 1.4–1.6, headings 1.1–1.25
`letter-spacing`	`0.08em`	Small positive for ALL-CAPS labels; slightly negative for large headings
`word-spacing`	`0.1em`	Rarely needed
`text-align`	`start` | `end` | `center` | `justify`	`start`/`end` follow writing direction; avoid `justify` without hyphenation (rivers: gaps lining up down the paragraph)
`text-transform`	`uppercase` | `capitalize`	Keep meaningful capitals in the HTML; screen readers read the source
`text-indent`	`2em`	First-line indent (books)
`text-decoration`	`underline wavy red`	Longhands: `-line`, `-style` (solid/double/dotted/dashed/wavy), `-color`, `-thickness`
`text-underline-offset`	`0.2em`	Moves the underline away from descenders
`white-space`	`nowrap` | `pre` | `pre-wrap`	Space collapsing and wrapping
`overflow-wrap`	`anywhere` | `break-word`	Break a long word/URL only if it would overflow (§10.10)
`word-break`	`break-all`	Break between any letters — harsh; mainly CJK (Chinese, Japanese, Korean) or code
`hyphens`	`auto`	Dictionary hyphenation; needs a `lang` attribute; Baseline since 2023
`text-wrap`	`balance` | `pretty`	Better line breaking (below)
`font-variant-numeric`	`tabular-nums`	Equal-width digits so numbers align in tables and prices
`max-inline-size`	`65ch`	The **measure** — line length

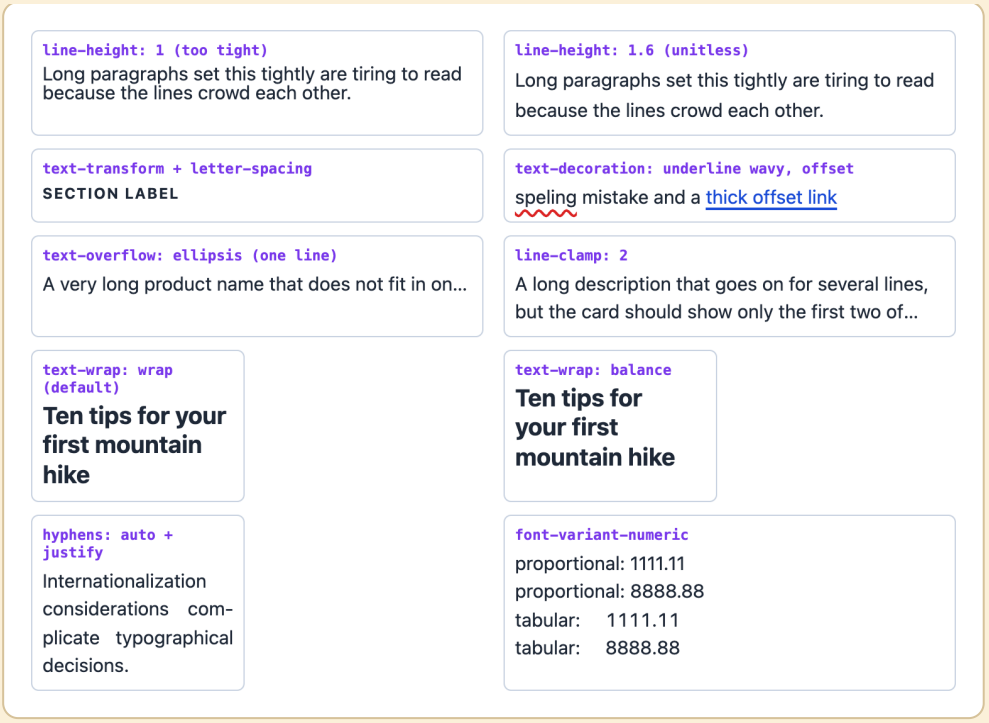


Figure 11.3 — `code/03_text.html`.

Why `line-height` should be unitless

`code/03_text.mjs` puts a 32px child in a 16px parent with `line-height: 1.5` vs `1.5em`:

```
line-height: 1.5   -> child (32px) line box 48px
line-height: 1.5em -> child (32px) computed 24px, line box 24px  (text taller than its line!)
proportional-nums width of "1" 6.73px, of "8" 9.36px
tabular-nums      width of "1" 9.22px, of "8" 9.22px
.caps letter-spacing computed: 0.96px (0.08em of 12px)
.balance text-wrap: balance
```

A unitless value is inherited as a **factor** and re-multiplied by each element's font size (32 × 1.5 = 48px). A length (`em`, `px`, `%`) is computed **once** on the parent (16 × 1.5 = 24px) and inherited as fixed pixels — bigger text overlaps its neighbours. The same output proves `tabular-nums`: every digit 9.22px wide.

Line length (the measure)

Comfortable reading needs roughly **45–75 characters per line**; `max-inline-size: 65ch` (`ch` ≈ width of "0") keeps paragraphs there at any width.

Better line breaking: `text-wrap`

- `text-wrap: balance` makes a short block's lines about equally long — no orphaned word under a long headline. Browsers only balance short blocks (it's expensive). **Baseline since May 2024.**
- `text-wrap: pretty` uses a slower paragraph algorithm, mainly avoiding a very short last line. Chrome and Safari 26, not Firefox — **not Baseline**; harmless as an enhancement.

Truncation

```css
/* One line with "…" — all three are required */
.title { white-space: nowrap; overflow: hidden; text-overflow: ellipsis; }

/* N lines with "…" — the prefixed syntax is the one that works in every browser */
.desc { display: -webkit-box; -webkit-box-orient: vertical; -webkit-line-clamp: 2; overflow: hidden; }
```

Unprefixed `line-clamp` is specified but not implemented; the `-webkit-` version is interoperable. Truncation hides content from sighted users — keep the full text reachable (a `title` tooltip is **not** enough for touch and keyboard users).

Trap seen while building Figure 11.3: in a `1fr` grid column a `nowrap` line makes the **column** grow, so no ellipsis appears. Fix: `minmax(0, 1fr)` (§9.10) or `min-width: 0` on the item.

Common mistakes

- `line-height` in `px` on `body` — every heading inherits the same fixed spacing.
- Light grey, thin, small text — fails contrast (11.4).
- Centre-aligned long paragraphs; all-caps paragraphs.
- Removing link underlines in body text: colour mustn't be the only link cue (WCAG 1.4.1) unless link/text contrast is ≥ 3:1 *and* another cue appears on hover/focus.

11.4 Colour and contrast

§6.8 covered notations (hex, `rgb()`, `hsl()`, `oklch()`, alpha, `currentColor`). This section is about choosing readable colours and deriving palettes from one brand colour.

Contrast ratio — what it is

WCAG's **contrast ratio** is `(L1 + 0.05) / (L2 + 0.05)`, with L1/L2 the **relative luminance** (perceived brightness, 0 black to 1 white) of the lighter/darker colour — from **1:1** to **21:1**. The computation (undo sRGB gamma, weight green most, blue least) and its implementation are in §3.9; `code/contrast.mjs` is the same code. (The printed WCAG 2.0 formula uses a 0.03928 threshold; newer text corrects it to 0.04045 — identical results for 8-bit colours.)

The thresholds (WCAG 2.2)

4.5:1 normal text (1.4.3 AA); **3:1** large text (≥ 24px, or ≥ 18.66px bold); 7:1 / 4.5:1 for AAA (1.4.6); **3:1** for non-text UI against adjacent colours — icons, input borders, focus rings, chart lines (1.4.11 AA). Logos and purely decorative or disabled elements are exempt.

`code/04_contrast.mjs` — by the formula, then using colours the browser computed in `code/04_contrast.html`:

```
#000000 on #ffffff: 21.00:1  AAA
#767676 on #ffffff: 4.54:1  AA
#777777 on #ffffff: 4.48:1  large/UI only
#1d4ed8 on #ffffff: 6.70:1  AA
#ffffff on #3b82f6: 3.68:1  large/UI only
#e2e8f0 on #0f172a: 14.48:1  AAA
#fcd34d on #ffffff: 1.44:1  FAIL
#fff on var(--brand)                          -> rgb(255, 255, 255) on oklch(0.55 0.2 260)  = 5.02:1 AA
#fff on color-mix(in oklch, brand 70%, black) -> rgb(255, 255, 255) on oklch(0.385 0.14 260)  = 10.04:1 AAA
brand on oklch(from brand 95% 0.03 h)         -> oklch(0.55 0.2 260) on oklch(0.95 0.03 260)  = 4.32:1 large/UI
only
#000 on oklch(from brand l c calc(h + 180))  -> rgb(0, 0, 0) on oklch(0.55 0.2 80)  = 4.25:1 large/UI only
```

(Four browser rows identical to formula rows are omitted.)

- **#767676** is the lightest AA grey on white; #777777 fails.
- White on a typical "blue-500" button (#3b82f6) is 3.68:1 — fine for large bold text, **not** normal labels; many design systems use a darker button shade for this reason.
- Yellow on white is nearly invisible (1.44:1).
- The complement (opposite hue) has the **same** OKLCH lightness (55%) as the brand, yet black on it gives 4.25:1 while white on the brand gives 5.02:1: OKLCH lightness is perceptual but *not* WCAG luminance — always check the actual pair.



Figure 11.4 — Contrast computed inside the page from the browser's own colours.

Deriving colours: `color-mix()` and relative colour syntax

Derive shades from one token instead of hard-coding them:

```
:root { --brand: oklch(55% 0.2 260); }
.darker     { background: color-mix(in oklch, var(--brand) 70%, black); }   /* 70% brand, 30% black */
.tint       { background: oklch(from var(--brand) 95% 0.03 h); }           /* keep hue, set L and C */
.complement { background: oklch(from var(--brand) l c calc(h + 180)); }    /* rotate the hue */
.faded      { background: rgb(from var(--brand) r g b / 50%); }            /* same colour, half opacity */
```

- `color-mix(in <space>, A p%, B)` — Baseline since 2023; mixing in `oklch`/`oklab` is more even than `srgb`.
- **Relative colour syntax** `<fn>(from <colour> …)` exposes the source's channels as keywords (`l c h`, `r g b`, `h s l`, `alpha`) to keep, replace or `calc()`. Baseline since September 2024 (newly available).

Other colour tools

- `currentColor` — the element's `color`; icons and borders that follow text.
- `accent-color: #1d4ed8` — tints checkboxes, radios, range sliders, progress bars. Not yet Baseline (Chrome for Android lacks it) but degrades gracefully.
- `color-scheme` and `light-dark()` — §10.5.

Accessibility beyond ratios

- **Never colour alone** (WCAG 1.4.1): add icon, text or pattern to "red = error"; underline links in text.
- About 1 in 12 men has a colour-vision deficiency, red/green the classic problem; DevTools (Rendering → Emulate vision deficiencies) simulates them.
- **APCA** (Accessible Perceptual Contrast Algorithm) is being explored for the future WCAG 3 — not a requirement today; audits and laws reference WCAG 2.2's ratio.

In the interview

30-second answer: "WCAG contrast is (L1 + 0.05)/(L2 + 0.05) using relative luminance; AA needs 4.5:1 for body text, 3:1 for large text and for UI like input borders and focus rings. I check real pairs with DevTools or a script — HSL or even OKLCH lightness doesn't guarantee it — and never rely on colour alone. For palettes I define a brand token in OKLCH and derive shades with `color-mix()` or relative colour syntax."

11.5 Backgrounds and gradients

The background properties

Property	Values	Notes
`background-color`	any colour	Painted under all image layers; always set one as a fallback under images
`background-image`	`url()`, gradients, `image-set()`	Comma-separated **layers**; the **first is on top**
`background-repeat`	`repeat` | `no-repeat` | `repeat-x` | `space` | `round`	Default `repeat`
`background-position`	`center`, `right 10px bottom 10px`, `30% 70%`	Where the image sits
`background-size`	`auto` | `cover` | `contain` | `200px 100px`	`cover` fills and crops; `contain` fits entirely
`background-origin`	`padding-box` | `border-box` | `content-box`	Box that positions the image
`background-clip`	`border-box` | `padding-box` | `content-box` | `text`	Box where painting stops
`background-attachment`	`scroll` | `fixed` | `local`	`fixed` is ignored on iOS, not Baseline, and costs a repaint every scroll
`background`	shorthand	`url(a.jpg) center / cover no-repeat, #e0f2fe` — `size` follows `position` after `/`

Gradients

A **gradient** is a browser-generated image — no download, crisp at any size, animatable only indirectly.

Function	Shape	Example
`linear-gradient()`	along a line	`linear-gradient(135deg, #f59e0b, #ef4444 50%, #7c3aed)`
`radial-gradient()`	circles/ellipses from a point	`radial-gradient(circle at 30% 30%, #fff 0 8%, #60a5fa 40%, #1e3a8a)`
`conic-gradient()`	around a point (like a clock hand)	`conic-gradient(#3b82f6 0 40%, #22c55e 0 70%, #f59e0b 0)` — a pie chart
`repeating-*-gradient()`	the pattern repeats	`repeating-linear-gradient(45deg, #fde68a 0 10px, #f59e0b 10px 20px)` — stripes

Colour stops are `colour [position] [position]`. Two stops at one position make a **hard edge** (`linear-gradient(to right, green 50%, grey 50%)` is a 50/50 split), and a `0` position after a larger one means "same as the previous stop" — how the conic pie gets sharp slices. Direction: `to right`, `to top left`, or an angle (`0deg` = up, `90deg` = right).

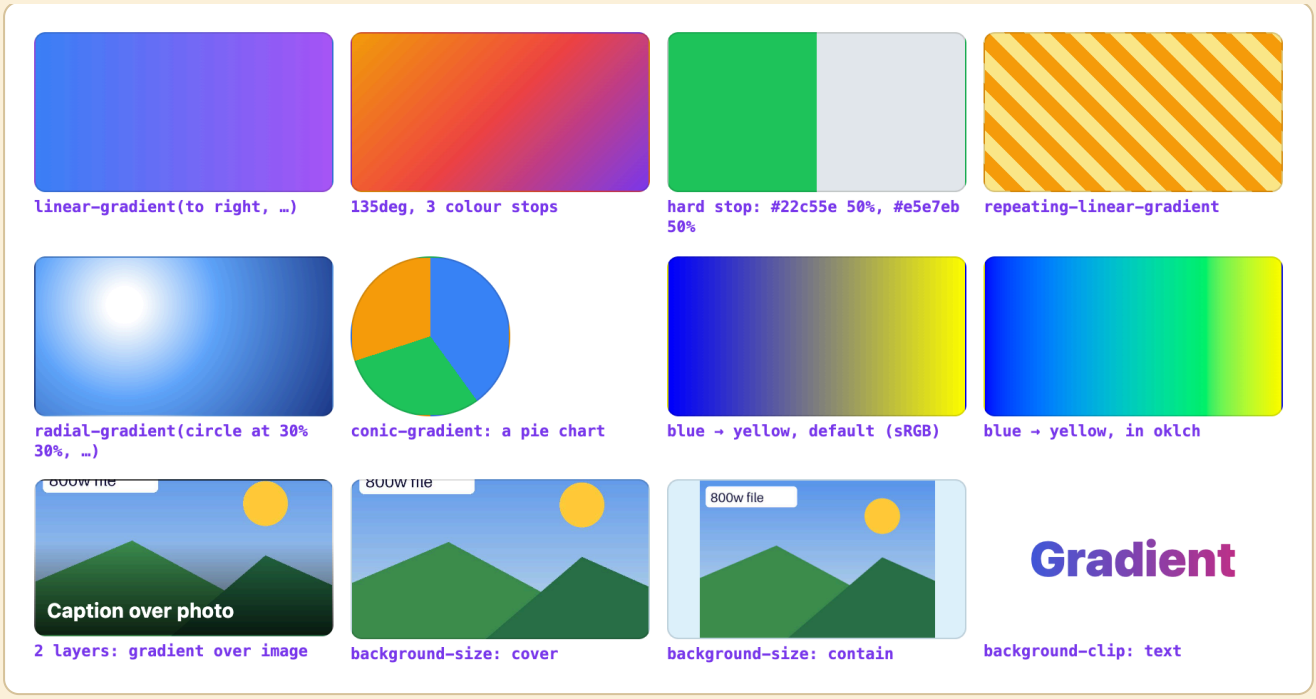


Figure 11.5 — `code/05_backgrounds.html`.

Interpolation colour space

Gradients blend in **sRGB** by default, so complementary colours pass through muddy grey. Since June 2024 (Baseline) you can pick the space: `linear-gradient(in oklch to right, blue, yellow)`. `code/05_backgrounds.mjs` reads screenshot pixels:

```
.srgb   at  50%: rgb(128, 128, 127)
.oklch  at  50%: rgb(0, 208, 188)
.hard   at  49%: rgb(34, 197, 94)
.hard   at  51%: rgb(229, 231, 235)
```

The sRGB midpoint of blue (0, 0, 255) and yellow (255, 255, 0) averages the channels to grey (128, 128, 127); in OKLCH the hue travels around the wheel through vivid teal. The `.hard` samples 2 % apart are the two pure stop colours.

Multiple backgrounds — a real use

Text over photos needs contrast: put a gradient layer **above** the image.

```css
.hero {
  background:
    linear-gradient(to top, rgb(0 0 0 / .75), transparent 60%),   /* layer 1: on top */
    url("hero.jpg") center / cover no-repeat;                     /* layer 2: below */
  color: #fff;
}
```

Gradient text

```
.gradient-text { background: linear-gradient(90deg, #2563eb, #db2777);
                 -webkit-background-clip: text; background-clip: text; color: transparent; }
```

Write both lines: web-features (the dataset behind Baseline) still lists unprefixed `background-clip: text` as not Baseline. Check contrast at the gradient's lightest point and keep a solid `color` fallback in forced-colours mode (an OS high-contrast setting that replaces page colours).

Common mistakes

- The gradient layer **after** the image — it's underneath and invisible.
- No `background-color` under a photo: if it fails, white text sits on white.

- Meaningful images as CSS backgrounds — no `alt`, not announced; use `<img>` (Chapter 4).
- Huge background photos on mobile — use `image-set()` (§10.8).

11.6 Borders, radius and outlines

Borders

`border: <width> <style> <colour>` on all sides, per side (`border-left`) or per aspect (`border-width: 1px 2px`). **The style is required** — initial value `none`, so `border: 2px #333` shows nothing (Chapter 7). Styles: `solid`, `dashed`, `dotted`, `double`, `groove`, `ridge`, `inset`, `outset`, `hidden`, `none`. Borders take layout space. `border-image` slices an image into a border — rarely used.

`border-radius`

Rounds the corners of the **border box** (clipping the background with it):

Value	Result
`8px`	all four corners
`24px 0 24px 0`	top-left, top-right, bottom-right, bottom-left (clockwise)
`50%` on a square	a circle (`aspect-ratio: 1` keeps it square)
`999px`	a **pill**: an oversized radius is reduced to fit, so the ends are exact semicircles at any height
`50% / 30%`	**elliptical** corners: horizontal radii / vertical radii

Percentages use the box's own width (horizontal radius) and height (vertical). Overlapping adjacent radii are scaled down proportionally. (`corner-shape` for squircles and bevels is Chromium-only in 2026.)

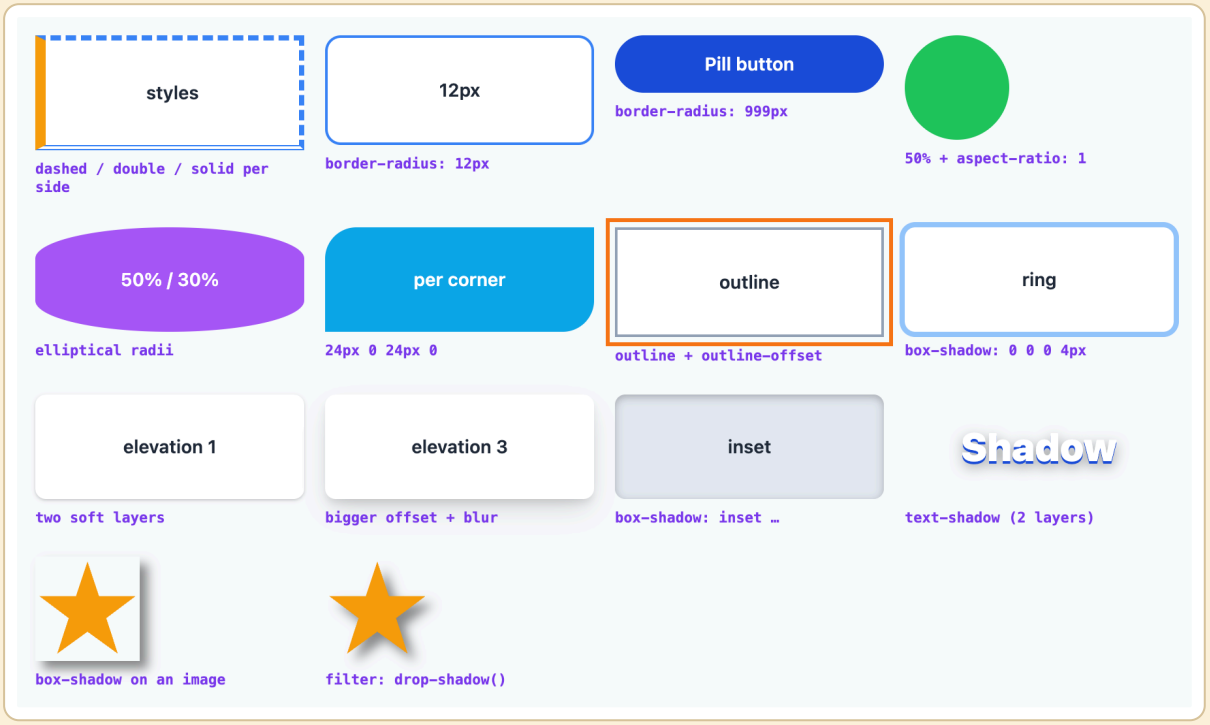


Figure 11.6 — `code/06_borders_shadows.html`.

Outline vs border vs ring

	`border`	`outline`	`box-shadow: 0 0 0 4px` ("ring")
Takes layout space	**yes**	no	no
Per-side control	yes	no	no
Follows `border-radius`	yes	yes (all current browsers since 2023)	yes
Offset	—	`outline-offset` (can be negative)	via extra layers
Shown in forced-colours mode	yes	**yes**	**no** (shadows are removed)
Main use	structure, dividers	**focus indicators**	decorative rings

`code/06_borders_shadows.mjs` confirms only borders change layout size (every box has `height: 80px`):

```
.styles   layout box 206x88  border-left 8px  outline none       shadow none
.outline  layout box 206x84  border-left 2px  outline 3px +4px   shadow none
.ring     layout box 206x80  border-left 0px  outline none       shadow rgb(147, 197, 253) 0px 0px 0px 4px
.elev3    layout box 206x80  border-left 0px  outline none       shadow rgba(0, 0, 0, 0.1) 0px 4px 6px -1px,
rgba(0,
.ellipse  border-top-left-radius: 50% 30%
```

So focus styles use `outline` — `:focus-visible { outline: 3px solid #2563eb; outline-offset: 2px; }` doesn't shift layout and survives Windows contrast themes. Never remove it without a replacement (§3.7). A popular pattern: a transparent outline plus a box-shadow ring, so forced-colours mode still gets the outline.

11.7 Shadows

`box-shadow`

```
box-shadow: [inset] <offset-x> <offset-y> [<blur>] [<spread>] <colour>, … ;
```

Part	Effect
offset-x / offset-y	Moves the shadow right/down (negative = left/up) — implies light direction
blur	Softness; 0 = hard edge
spread	Grows (positive) or shrinks (negative) the shadow before blurring
`inset`	Drawn inside the box (pressed/sunken look)
multiple, comma-separated	First on top; layer a tight dark shadow and a large soft one for realistic depth

Elevation systems (Material Design and most component libraries) show how "high" a surface floats with increasingly large, soft, offset shadows:

```
.elev1 { box-shadow: 0 1px 2px rgb(0 0 0 / .12), 0 1px 3px rgb(0 0 0 / .08); }
.elev3 { box-shadow: 0 4px 6px -1px rgb(0 0 0 / .1), 0 10px 20px -5px rgb(0 0 0 / .15); }
```

The negative spread keeps the large soft shadow from peeking out at the sides. Shadows don't affect layout and paint outside the box (so a **parent's** `overflow: hidden` clips them).

`text-shadow`

`text-shadow: <x> <y> [<blur>] <colour>, …` — no spread, no inset. For legibility on busy images (a soft dark halo) or display effects; keep body text shadow-free.

`filter: drop-shadow()`

`box-shadow` follows the **box**; `filter: drop-shadow(x y blur colour)` follows the element's **alpha shape** (its non-transparent pixels) — the star in Figure 11.6 gets a star-shaped shadow, as would a transparent PNG logo. No spread or inset, and costlier to render.

Performance

Large blurred shadows are expensive to paint, and **animating** `box-shadow` repaints every frame. To animate elevation, put the bigger shadow on a pseudo-element and animate its `opacity` (Chapter 12).

11.8 Filters, blend modes, clipping and masks



Figure 11.7 — `code/07_filters_blend.html`.

`filter`

Applies image-processing functions to an element **and everything inside it**, in the order written:

Function	Example	Effect
`blur()`	`blur(3px)`	Gaussian (smooth, even) blur
`brightness()`	`brightness(1.4)`	< 1 darker, > 1 brighter
`contrast()`	`contrast(1.8)`	
`grayscale()`	`grayscale(1)`	0–1 (or %)
`hue-rotate()`	`hue-rotate(120deg)`	Rotates hues
`invert()`	`invert(1)`	
`opacity()`	`opacity(.5)`	Like the `opacity` property
`saturate()`	`saturate(3)`	
`sepia()`	`sepia(1)`	
`drop-shadow()`	11.7	
`url(#svgFilter)`		Any SVG filter

Baseline widely available since 2016. Uses: greyscale logos that colour on hover, dimming images in dark mode, disabled states.

`backdrop-filter`

Filters what's **behind** the element (which must be partly transparent) — the "frosted glass" of macOS/iOS panels: `background: rgb(255 255 255 / .3); backdrop-filter: blur(6px) saturate(1.4);`. Baseline since September 2024 (Safari 18 dropped the `-webkit-` requirement). Check text contrast against the *worst* possible background and provide a more opaque fallback for `prefers-reduced-transparency` where supported.

Blend modes

- `mix-blend-mode` — how the element's pixels combine with content **behind** it.
- `background-blend-mode` — how an element's **own background layers** combine; used for duotone images (recoloured in two tones).
- `isolation: isolate` — a new stacking context so blending stays inside a group (§7.8).

Per channel, values 0–1 (a = top, b = backdrop): `multiply` = a·b (always darker; white disappears), `screen` = 1 − (1−a)(1−b) (always lighter; black disappears), `overlay`, `darken`/`lighten`, `difference` = |a − b|, `luminosity`/`color`/`hue`/`saturation` (mix HSL-like components). `code/07_filters_blend.mjs` blends #ef4444 over sky blue and compares the screenshot pixel with the formula:

```
multiply   by hand rgb(94, 40, 63)  browser rgb(94, 40, 63)
screen     by hand rgb(245, 178, 240)  browser rgb(245, 178, 240)
difference by hand rgb(139, 82, 167)  browser rgb(139, 82, 167)
```

Clipping and masking

- `clip-path` hides everything outside a shape: `circle(40% at 50% 50%)`, `ellipse()`, `inset(10px round 8px)`, `polygon(0 0, 100% 0, 100% 75%, 0 100%)`, `path("…")`, or SVG `url(#clip)`. Hard-edged; clicks outside pass through. Baseline since 2021 (animatable between shapes with equal point counts).
- `mask-image` uses an image's **alpha** (or luminance) as a soft mask: `mask-image: linear-gradient(to right, #000 40%, transparent)` fades an element out. Unprefixed masks Baseline since December 2023.

Side effects to remember

`filter`, `backdrop-filter`, `mix-blend-mode`, `clip-path`, `mask` or `opacity < 1` **create a stacking context** (§7.8); a non-`none` `filter` or `backdrop-filter` also becomes the **containing block for** `position: fixed` **descendants** — a "fixed" header inside a filtered parent scrolls with it. Filters are GPU work; large blurred areas on low-end phones can drop frames.

11.9 `object-fit` and `object-position`

What it is

For **replaced elements** (content from an external source, not CSS) — `<img>`, `<video>`, `<canvas>`, `<iframe>` content — `object-fit` decides how the content fills the element's box when aspect ratios differ: the `background-size` of images. Baseline widely available (2020).

Value	Scale factor	Result
`fill` (default)	separate x and y	Stretched — **distorted**
`contain`	min(box/content) per axis	Whole image visible, **letterboxed** (empty bars)
`cover`	max(box/content) per axis	Box filled, overflow **cropped**
`none`	1	Natural size, centred, cropped
`scale-down`	the smaller of `none` and `contain`	Never enlarges

`object-position` (default `50% 50%`) chooses which part stays visible when cropping, or where a letterboxed image sits — like `background-position` (`0% 50%` keeps the left edge, vertically centred).

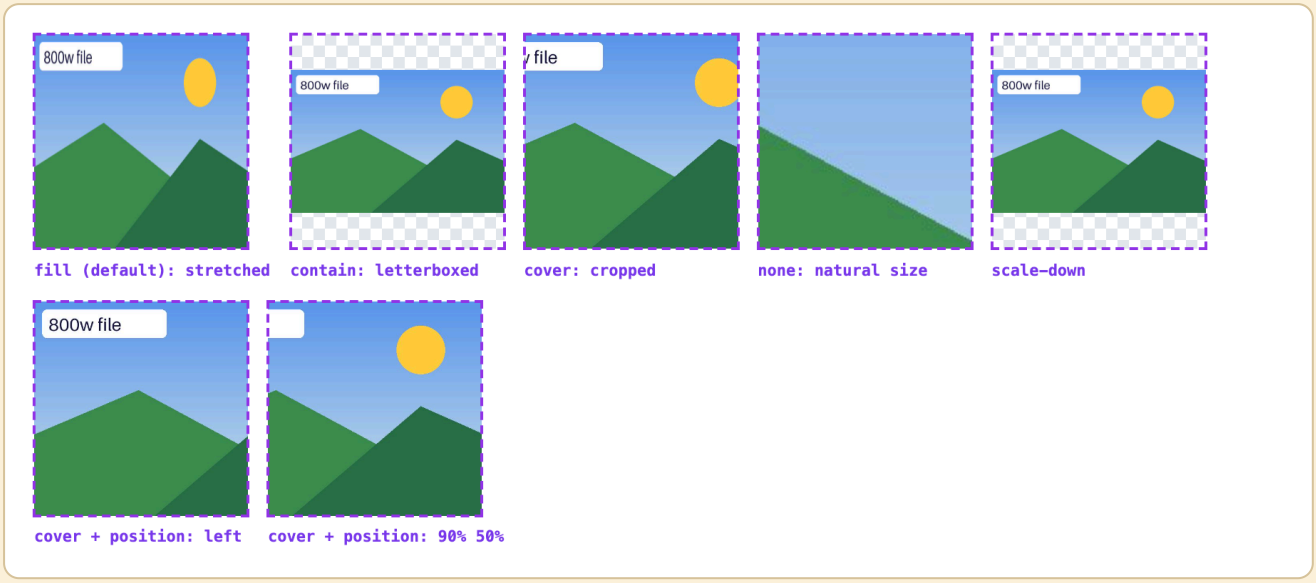


Figure 11.8 — `code/08_object_fit.html`. Dashed purple = the element's box; checkerboard = empty space.

`code/08_object_fit.mjs` prints each box's computed values, then the maths by hand from the natural size:

```
fill         box 150×150  natural 800×533  object-fit fill       object-position 50% 50%
...
cover-right  box 150×150  natural 800×533  object-fit cover      object-position 90% 50%

contain: scale = min(150/800, 150/533) = 0.1875 -> drawn 150×99.94 (bars above and below)
cover:   scale = max(150/800, 150/533) = 0.2814 -> drawn 225.14×150, 75.14px cropped horizontally
object-position 90% 50% on cover: image x offset = (150 - 225.14) × 0.9 = -67.63px
scale-down: the smaller of none (scale 1) and contain (0.1875) -> same as contain here
```

A percentage position aligns that percentage point of the image with the same point of the box, so offset = (box − image) × percentage — the `background-position` rule. (Example: 800×400 in 200×200 → `cover` scale 0.5 → 400×200, 200px cropped; `contain` scale 0.25 → 200×100, 50px bars.)

Real-world use

- **Card thumbnails / avatars** of unpredictable shape: `width: 100%; aspect-ratio: 4 / 3; object-fit: cover;` (+ `border-radius: 50%` for avatars).
- **Product photos** that mustn't be cropped: `object-fit: contain` on a neutral background.
- **Hero video**: `<video autoplay muted loop playsinline>` + `object-fit: cover` for a full-width banner.

- **Grid cells**: images don't stretch by default (§9.10); `width: 100%; height: 100%; object-fit: cover` fills them.
- When cropping could cut off the subject (faces), set `object-position` per image, e.g. inline from a CMS (content management system) focal point.

Common mistakes

- Only `width` on an `<img>` — the box needs both dimensions (via `height` or `aspect-ratio`) differing from the image's ratio for `object-fit` to matter.
- `object-fit` on a `<div>` — replaced elements only; use `background-size`.
- `cover` cropping important content (text in images).

Chapter 11 TL;DR

- **Fonts**: stacks end with a generic family (`system-ui` stack = 0 bytes); `@font-face` + **WOFF2**; one **variable font** (`font-weight: 100 900`) replaces many files; missing faces are **synthesized** unless `font-synthesis: none`.
- **Loading**: `font-display` `block` (invisible ≤3 s) / `swap` / `fallback` / `optional` (≈100 ms, then never); preload 1–2 fonts **with** `crossorigin`, self-host, subset with `unicode-range`, `size-adjust` the fallback.
- **Text**: `rem` sizes, **unitless** `line-height`, 45–75ch measure, `text-wrap: balance` (Baseline 2024), ellipsis = nowrap + hidden + `text-overflow`, `-webkit-line-clamp`, `tabular-nums`, `hyphens: auto` needs `lang`.
- **Contrast**: (L1+0.05)/(L2+0.05); AA 4.5:1 text, 3:1 large text and UI; #767676 is the lightest AA grey on white; never colour alone; derive shades with `color-mix(in oklch, …)` / `oklch(from var(--c) l c h)` and still check each pair.
- **Backgrounds**: first layer on top; `cover`/`contain`; linear/radial/conic/repeating gradients, hard stops, `in oklch` avoids grey midpoints.
- **Borders/shadows**: border style required; `999px` pills, `a / b` ellipses; focus = `outline` (no layout, survives forced colours); layered `box-shadow` for elevation; `drop-shadow()` follows alpha; don't animate shadows directly.
- **Effects**: `filter`, `backdrop-filter` (Baseline 2024), blend modes (multiply = a·b, screen, difference), `isolation`, `clip-path`, `mask-image` — all create stacking contexts; filters also capture `position: fixed`.
- **`object-fit`**: fill / contain (min scale) / cover (max scale) / none / scale-down, + `object-position`.

Next: Chapter 12 — Transitions, Transforms and Animation.